

# HDB Technical Test for Senior Data Engineer

## **Introduction**

This is a technical test to assess the engineering and solutioning competencies for a senior data engineer role in HDB.

Duration: **1 week** upon receiving the assignment

## **Part 1: Developing Data Pipelines**

### **Background**

The Data Engineering Team is tasked to provide data cleaning and transformation services to the Data Science Team to aid them in their analysis of resale flat prices. You will need to design an ETL pipeline using datasets from **January 2012 to December 2016**.

All tasks should be performed in a programmatic fashion where possible. Avoid manual interface interactions and hardcoding where possible.

### **Dataset Extraction:**

The required dataset can be found at <https://data.gov.sg/collections/189/view>. Your ETL pipeline should process the data file as it is, without any manual modifications (e.g. manually deleting any rows or columns).

### **Data Quality Requirements:**

1. Combine the datasets into a single master dataset. The combined data dataset should contain all the attributes found in all files.
2. Perform Data Profiling on the dataset. You may code your own profiling rules or leverage on open-source data quality frameworks.
3. Design data validation rules to validate the following fields (i.e. Date, Town, Flat Type, Flat Model, storey\_range) based on the statistical properties of this master dataset.
4. Assume HDB lease is 99 years old, recompute remaining lease as of today. Remaining lease should be rounded down to Years and Months.
5. You may assume the composite key for the dataset is all columns except the resale price. If all columns have the exact same value, except for the resale price (i.e. duplicated key), take the higher price and **discard** the lower price into the “failed” dataset (if any).
6. Identify any potentially anomalous resale price based on any heuristics you deem appropriate. Please document your heuristic and assumptions in your documentations.

7. Any additional data cleaning/validation rules that you may deem appropriate. The actual dataset may not have anomalous data, but mechanisms to identify such data can be demonstrated in your assignment.

### Data Transformation Requirements:

1. Taking the cleaned data from the previous step, create a new column called “Resale Identifier”. The “Resale Identifier” is derived from:
  - First character is “S”.
  - Next 3 digits is the first 3 digits of the *block* column, after removing any characters. In the event the block column has less than 3 digits, prepend with the appropriate number of zeros in front. (E.g. if block number is “19”, digits will be “019”)
  - Next 2 digits is derived by: Taking the 1<sup>st</sup> and 2<sup>nd</sup> digit of the average resale price, group by year-month, town and flat\_type. For instance, if the average Ang Mo Kio, 2 Room Flats in Jan 2017 is \$230000, the 2 digits is “23”.
  - The last two digits is the *month* of the current entry (e.g. if the month is 2012-01, the last two digits is 01).
  - The final character is the first character of the *town*. E.g. Ang Mo Kio, the character is “A”.
2. If there are any duplicate records, take the higher price and **discard** the lower price one.
3. Hash this identifier column using an irreversible hashing algorithm, while preserving its uniqueness. Explain the hashing algorithm that you adopted in your documentations.

### Data Output Requirements:

1. You may create any number of tables or view to aid you in the ETL process.
2. There are 3 mandatory groups of output files to be produced:
  - **Raw**: Contains the raw data files as-it-is.
  - **Cleaned**: Dataset that passes the data quality requirements
  - **Transformed**: Dataset that went through the transformation requirements
  - **Failed**: Dataset that contains records that were removed (E.g. Duplicated Records, Data that does not comply with any validation rules)
  - **Hashed**: Cleaned Data + Hashed Identifier Column

### Submission Requirements:

1. ETL Pipeline to be developed in **Python**. You are free to design your own pipeline in any ways deemed appropriate.
2. A Jupyter Notebook file with input files to be uploaded to a Git Repository, with in-line comments and documentations to guide us to execute the codes and produce the intended output.
3. Ensure that your code adheres to engineering best practices, with a focus on code quality, explainability, maintainability, and robustness.

## **Part 2: Architecting Data Ingestion & Data Exploitation Solution Patterns**

### **Background**

After developing the pipeline, there is a need to operationalise the pipeline, which include designing the architecture to support Data Ingestion and Data Exploitation, thereby enabling end-to-end flow of data to support analysis of the dataset.

Assume that HDB's data platform is built on AWS, and the target data store is AWS S3, design solution architectures that would support the requirements. Use the AWS architecture icon pack to aid in your architecture diagrams (<https://aws.amazon.com/architecture/icons/>).

The design should take into consideration security, scalability and performance. You may make necessary assumptions in your design and document them down accordingly.

The objective of this test is on architectural design and solutioning. It is not necessary to produce actual IAM policies. State the assumptions that you used in your design.

### **Data Ingestion Requirements**

The architecture should support:

1. Batch data ingestion from data.gov.sg. The design should support large files transmission (e.g. > 100 MB)
2. Data.gov.sg endpoint is Public-Facing, while HDB's Data Platform resides in Private VPCs. The design should factor in network/system component segmentations as part of security considerations.

### **Data Exploitation Requirements**

The data science team is using Tableau on AWS for their analysis. The architecture should support:

1. Data Integration via Athena Driver ([https://help.tableau.com/current/pro/desktop/en-us/examples\\_amazonathena.htm](https://help.tableau.com/current/pro/desktop/en-us/examples_amazonathena.htm))
2. Usage of the Athena Driver requires calling AWS Athena Public Endpoint. The integration design should factor in network security by making the traffic flow private where possible.

### **Submission Requirements:**

Output could be in Power Point or using draw.io or equivalent tool, converted to PNG for submission purpose.

Together with Part 1, upload the architectural designs to the same Git Repository and configure it to be publicly accessible. Do not upload this test document into the Git Repository.